

Incentive-Aware Partitioning and Offloading Scheme for Inference Services in Edge Computing

TaeYoung Kim[✉], Chang Kyung Kim[✉], Seung-seob Lee[✉], and SuKyoung Lee[✉], *Member, IEEE*

Abstract—Owing to remarkable improvements in deep neural networks (DNNs), various computation-intensive and delay-sensitive DNN services have been developed for smart IoT devices. However, employing these services on the devices is challenging due to their limited battery capacity and computational constraints. Although edge computing is proposed as a solution, edge devices cannot meet the performance requirements of DNN services because the majority of IoT applications require simultaneous inference services, and DNN models grow larger. To address this problem, we propose a framework that enables parallel execution of partitioned and offloaded DNN inference services over multiple distributed edge devices. Noteworthy, edge devices are reluctant to process tasks due to their energy consumption. Thus, to provide an incentive mechanism for edge devices, we model the interaction between the edge devices and DNN inference service users as a two-level Stackelberg game. Based on this model, we design the proposed framework to determine the optimal scheduling with a partitioning strategy, aiming to maximize user satisfaction while incentivizing the participation of edge devices. We further derive the Nash equilibrium points in the two levels. The simulation results show that the proposed scheme outperforms other benchmark methods in terms of user satisfaction and profits of edge devices.

Index Terms—Incentive, DNN Partitioning, offloading, scheduling, utility, energy, inference delay, game theory.

I. INTRODUCTION

WITH the proliferation of smart devices, a growing number of deep neural network (DNN) services, including computer vision, machine translation, natural language processing, and many other services, are being applied in various emerging IoT systems [1], [2]. These services are typically computation-intensive and delay-sensitive, whereas smart devices are usually computation-constrained and battery-limited. Therefore, edge computing has been introduced to provide them with computing resources to meet their demand for high computation performance [3], [4], [5], [6].

In edge computing, computation offloading has been widely recognized as a key enabling technology [7]. Hence, many

studies have explored different ways to efficiently apply this technique to DNN-based applications, aiming to provide timely and resource-efficient inference services [3], [8]. Yet, compared to cloud computing, the resources provided by edge devices are limited, leading to a failure in meeting the performance requirements of DNN services.

Computation offloading approaches at the edge can be divided into two main categories: full and distributed offloading. Full offloading, explored extensively in previous studies [9], involves moving the entire computational process of DNN models to an edge device. However, as more IoT applications demand simultaneous DNN inference services and DNN models grow larger [3], [8], [10], this single-device offloading approach faces significant challenges [11], [12]. In response, recent works prioritize distributed offloading, which partitions the computation across multiple edge and user-end devices, thus effectively managing large computational tasks [1], [2], [3], [6], [8], [10], [12], [13], [14], [15], [16], [17]. For distributed offloading in a practical edge computing environment, it is crucial to 1) enable parallel execution of DNN inferences to provide timely edge inference and 2) design effective incentive mechanisms to stimulate resource sharing between edge devices.

Timely inference at the edge is essential in numerous use cases including security, surveillance, healthcare, and autonomous driving (e.g., to inspect visitors' faces, or to detect heartbeat frequency) [18], [19]. Therefore, to reduce the waiting time for such time-critical inference requests, a computation offloading framework should allow parallel execution of DNN inferences (e.g., to prevent head-of-line blocking) [20], [21]. Specifically, the offloading framework must consider dependency between DNN layers and operations in parallel executions, while ensuring the correct order of execution, reducing communication overhead, and utilizing edge computing resources more effectively. Although some previous works consider task dependency [10], [16], [22], none of them consider the parallel execution in multiple edge devices which is much more complex than traditional DNN partitioning and offloading.

In practice, edge servers/devices are reluctant to cooperatively run offloaded computation tasks due to high energy consumption. Therefore, the offloading framework should consider incentive mechanisms to stimulate resource sharing between edge servers/devices. Although many studies have designed incentive mechanisms for edge computation offloading [7], [23], [24], [25], these mechanisms are applicable only to general computation tasks and not to DNN inferences. For instance, the task partitioning techniques presented in [18], [26] assumed that the

Manuscript received 29 June 2023; revised 22 January 2024; accepted 22 January 2024. Date of publication 26 January 2024; date of current version 8 August 2024. This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government (MSIT) under Grant 2022R1A2B5B01001683. (Corresponding author: SuKyoung Lee.)

TaeYoung Kim, Chang Kyung Kim, and SuKyoung Lee are with the Department of Computer Science, Yonsei University, Seoul 03722, South Korea (e-mail: tykim92@yonsei.ac.kr; ckim48@yonsei.ac.kr; sklee@yonsei.ac.kr).

Seung-seob Lee is with Yale University, New Haven, CT 06520 USA (e-mail: seung-seob.lee@yale.edu).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2024.3359148>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2024.3359148

computation tasks can be arbitrarily partitioned, offloaded, and executed in parallel, whereas the stronger dependency between operations and layers in DNNs renders arbitrary partitioning inapplicable.

These problems and challenges motivate us to design an incentive-aware partition and offloading scheme for DNN inference services in the edge computing environment. In this study, we first designed a framework that enables parallel execution of partitioned/offloaded DNN inferences considering task dependencies and communication delays between them. To model the practical, profit-driven behavior of edge servers and devices, the framework employs game theory, in particular, a two-level Stackelberg game-based model. Finally, atop this framework, we propose a DNN partitioning and offloading algorithm that considers inference delay, energy consumption, and incentives offered by edge users. Thus, in the proposed framework, the optimal scheduling with partitioning strategy is determined such that users' satisfaction in terms of inference delay is maximized, while the incentive purpose is achieved by setting optimal prices for edge devices.

In summary, the contributions of this paper are as follows:

- We propose an Incentive-aware Partitioning and Offloading (IPO) scheme to encourage the edge devices to participate in the DNN partition and distributed offloading. Specifically, we model the edge computing environment as a computing resource market, where the DNN inference service requesters (SRs) pay for the use of computing resources provided by service providers (SPs) willing to offer their computing resources for achieving profit. We then formulated the problem of maximizing the utility of each edge device in terms of inference delay and satisfaction for SRs, and energy consumption and profits for SPs.
- We designed a framework that enables parallel execution of partitioned DNN inference services across multiple edge devices. The proposed framework facilitates edge users to independently partition and offload the DNN inference tasks across multiple edge devices by considering task dependencies and communication delays between the partitioned tasks, aiming to maximize the utilities of SPs and SRs. For the parallel execution of the tasks, we not only model the piece-wise based DNN partitioning considering the computing resource and energy consumption of each edge device, but also develop the scheduling model considering the task dependency of sub-tasks and the deadline of the service.
- To solve the DNN partitioning and offloading problem, we design the interaction process between SRs and SPs as a two-level Stackelberg game. This hierarchical approach better reflects many real-world scenarios than other game theory methods such as auction-based game theory [25], [27]. Then, the optimal game strategies are designed to optimize DNN partitioning and scheduling decisions of SRs, where these SR decisions dynamically change according to the price strategies of SPs and vice versa. We conducted a theoretical analysis to illustrate the existence and uniqueness of Nash equilibrium in the proposed game.

Then, we conduct simulation experiments to support our theoretical analysis.

- Finally we validate the effectiveness of the proposed framework and incentive mechanism through simulation. The numerical results not only demonstrate that the proposed algorithm converges within 50 iterations, but also show that it outperforms other benchmark algorithms in terms of utility.

The rest of this paper is organized as follows: After reviewing the related works in Section II, Section III describes the system model including the overall process of IPO scheme, the DNN partition model, and the scheduling delay model. Further, Section IV formulates the incentive mechanism based on the two-stage Stackelberg game. Then, Section V provides the proposed algorithm to find Nash equilibrium. In Section VI, the conducted performance evaluation is detailed and the simulation results are provided. Finally, Section VII concludes this article.

II. RELATED WORK

A. DNN Task Offloading in Edge Computing

Although DNN-based applications have gained popularity in IoT networks, it is challenging to execute such DNN inference tasks on resource-constrained user devices. A feasible solution is to offload the DNN inference tasks to the near edge devices or servers.

Some research works [28], [29], [30] in this field investigated the DNN task offloading in edge computing to minimize the inference delay as well as the energy consumption resulting from the execution of DNN tasks on edge servers. Fan et al. [28] jointly considered the DNN model deployment, offloading, and resource allocation to minimize the inference delay. Qu et al. [29] proposed a dynamic DNN offloading framework that considers the heterogeneity of computing and radio resources available on the edge server to minimize the inference delay and energy consumption. Yoon et al. [30] designed a DNN-based object detection offloading framework to choose the optimal edge servers for processing heavy and lightweight workloads to near-edge servers by considering their computing power and network bandwidth in a real WLAN environment. Prior research has demonstrated that offloading DNN inference services to optimal edge servers can effectively reduce inference delay. However, even though the edge servers have more abundant computing resources than the user device, a single edge server is inadequate for performing computation-intensive tasks to meet the requirement of services.

To address this challenge, some prior studies suggested parallel execution of computation-intensive tasks in multiple edge servers [20], [21]. Chen et al. [21] designed a parallel offloading strategy based on Lyapunov optimization, which finds the optimal decision for delay-sensitive and computation-intensive tasks. Huang et al. [20] proposed a distributed task offloading strategy for IoT devices in the overlapping coverage areas of several base stations (BSs) deciding how to divide and distribute the tasks into multiple edge servers, while minimizing the delay. Although these earlier works proposed different distributed offloading strategies to provide computation-intensive tasks with

minimal delay, they did not consider the DNN inference tasks, which require DNN partitioning. To provide DNN inference services under strict performance requirements, it is crucial to consider both parallel execution and DNN partitioning to improve the resource utilization of edge servers and accelerate the processing of DNN inference services.

B. Partitioning and Offloading DNN Model

Due to the high computation required to support multiple simultaneous DNN models and the large size of models, traditional computation offloading faces challenges in providing timely edge inference as it offloads the entire DNN computation. To address these challenges, recent studies have focused on distributed offloading, which partitions the intensive DNN computation across multiple edge and user-end devices [1], [10], [16], [22].

Zeou et al. [1] proposed the fused-Layer-based DNN model parallelism method, which converts a DNN layer to several smaller layers, to increase partial computation offloading flexibility and to reduce the inference delay. Fand et al. [10] considered both radio and computing resources of edge devices, and suggested a DNN partition mechanism that minimizes the processing delay of DNN inference task. Shi et al. [16] considered the scenario where the user requests a DNN inference task with the partitioned DNN model from the edge device to improve the performance of batch processing. Chen et al. [22] predicted the expected inference delay and computation cost for each layer to determine which DNN model layers are offloaded to the edge device.

However, these studies do not consider the execution of simultaneous DNN service requests from the devices, which can cause long inference delays. Therefore, several other studies have investigated collaborative computation among multiple edge devices to effectively handle multiple DNN services [3], [8], [11], [17]. He et al. [17] optimized the deployment of DNN partitions at the multiple edge devices to minimize the inference delay. Lin et al. [11] mapped the partitioned DNN subgraphs into different computing processors to improve the parallelism and reduce the delay of the DNN inference services. Zeng et al. [3] proposed a distributed DNN computing system that orchestrates cooperative DNN inference over multiple edge devices. Kim et al. [8] designed a DNN partition placement and resource allocation strategy that considers different processing powers, memory, and battery levels for heterogeneous IoT devices.

The studies discussed above successfully reduced energy consumption and inference delay through the partitioning of DNNs across multiple edge servers. However, these studies have not considered the self-interest of individual edge devices, which may be unwilling to share their resources without any rewards.

C. Incentive Mechanisms for Task Offloading and Scheduling

Since self-interested edge devices are unwilling to provide their resources, some previous studies investigated the incentive mechanism for DNN task offloading in edge computing [7], [23],

[24], [25], [26], [27], [31]. Lie et al. [23] developed an auction-based dependent task offloading mechanism with the problem of valuation maximization in the trade of computing resources satisfying delay requirements for the task. Xue et al. [24] used auction theory to investigate the incentive decisions of a service provider for DNN inference task scheduling. Hou et al. [32] proposed an online incentive-driven task allocation scheme for collaborative computing among edge servers and devices. Tang et al. [25] suggested the distributed pricing strategy based on the Stackelberg game theory in a scenario of multiple autonomous vehicles and multiple edge servers. Chen et al. [31] studied a game theory-based video analytics offloading algorithm for selecting the optimal edge server to minimize the delay in video analytics. Diamanti et al. [27] studied a joint communication and computing resource allocation problem that is designed between the edge server and the users in the form of a Stackelberg game with the consideration of heterogeneity in terms of applications' multidimensional distinctive features including their delay tolerance/sensitivity.

Baranwal et al. [33] designed a task allocation scheme with the incentive mechanism for collaborative computing among edge servers and IIoT devices using a blockchain mechanism without involving a trusted third party. Deng et al. [26] designed a fully distributed edge network architecture that leverages the blockchain-based incentive mechanism as a two-level Stackelberg game to overcome the limitation of decisions being made by a central server.

The above studies have investigated incentive-based task offloading, scheduling, or resource allocation to encourage edge devices to join computing and maximize their profits. Nevertheless, none of these works apply the incentive mechanism for DNN partition with scheduling considering the task dependency for parallel execution.

Motivated by the previous studies discussed above, we investigate the incentive-aware parallel execution framework for edge computing. To the best of our knowledge, this work is the first to develop a parallel execution framework that partitions with scheduling the DNN inference services while simultaneously considering the incentive mechanism to encourage edge devices to voluntarily participate in the computation. Specifically, we formulate the problem of maximizing the utility in terms of inference delay, energy consumption, and profits. Then, we develop a two-level Stackelberg game model for the strategic interaction between SRs and SPs. We design the game strategies for both SR and SP involve adjusting the price of computing resources based on the availability of resources in SPs. Then the DNN partitioning and offloading decisions are made by SRs based on the adjusted price. Simulation results show that, compared to other benchmark methods, the proposed approach shows lower inference delay and energy consumption while generating higher profits for edge devices in the system.

III. SYSTEM MODEL

In this section, we first describe the main considered elements of the system architecture and detail their interaction. Then, we provide the details of models developed for DNN partitioning

TABLE I
LIST OF MAIN NOTATIONS

Notation	Description
$\mathcal{D}$	Set of EDs
$\mathcal{N}$	Set of inference tasks
$\mathcal{L}_n$	Set of layers
$a_{i,n}$	Partition assigned to ED i
$s_{i,n,l}$	Boundary rows of output data from the l -th layer at ED i
$x_{i,n}$	Proportion that the $a_{i,n}$ partition covers
$b_{i,j}$	Transmission rate from ED i to ED j
τ_n	Deadline for task n
t_n	Time when the CNN task n is requested
$t_{i,n,l}^{(s)}, t_{i,n,l}^{(f)}$	Time when the layer l starts and finishes processing for task n
T_n	Total delay for task n
$E_{i,n}^{(p)}$	Energy consumption for processing task n at ED i
$E_{i,n}^{(tr)}$	Energy consumption for transmitting padding data
$U_{i,n}$	Utility of SP i of task n
$\tilde{U}_n$	Utility of SR for the task n
$p_{i,n}$	Unit price charged by SP i for task n
$g_{i,n}$	Degree of satisfaction of SR's

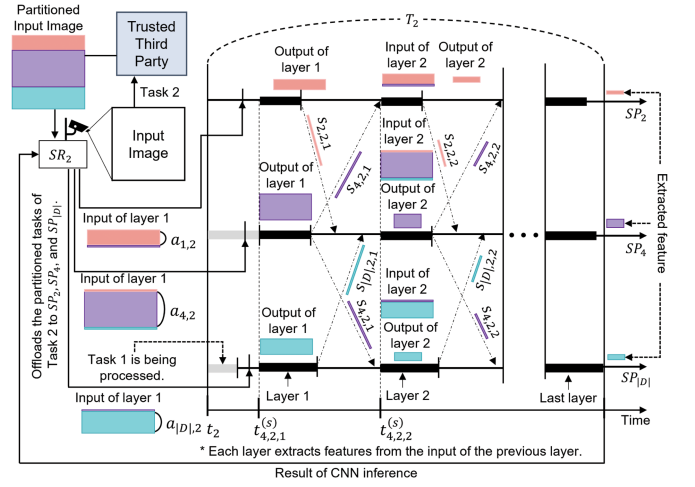


Fig. 2. Example of feature extraction process for the partitioned tasks.

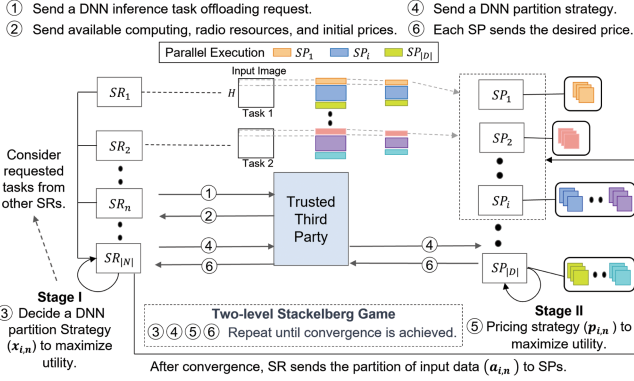


Fig. 1. Overview of the framework for IPO scheme.

and scheduling delay. The main symbols used in this paper are summarized in Table I.

A. Overall Design of Proposed Framework

The system architecture of our proposed framework contains a set $\mathcal{D}$ of edge devices (EDs) $i \in \mathcal{D}$, a set $\mathcal{N}$ of DNN inference tasks $n \in \mathcal{N}$ from user devices, and one trusted third party which can be a base station or WiFi-router [34].

The user requesting a DNN inference service (i.e. face detection for a smart surveillance camera system) is a *service requester* (SR), while the EDs that can provide its computing resources for DNN inference play the role of *service provider* (SP). The primary function of a trusted third party is to facilitate the interaction between SRs and SPs while each SR and SP make the DNN partition strategy and price decision, respectively, in a distributed manner.

As depicted in Fig. 1, when the SR requires a DNN inference service, the following steps occur: ① SR sends a DNN service request to the trusted third party. ② In response, the trusted third party provides the available radio and computing resources of registered SPs, along with their prices. ③ Based on the received

information, the SR independently determines how to purchase the computing resources and decides DNN partition strategy that can maximize its utility in terms of delay and payment amount. ④ Then, SR sends the decided partition strategy to the trusted third party. Subsequently, the trusted third party forwards the received partition strategy to the registered SPs. ⑤ Then, each SP engages in a competitive process to offer its computing resources at the most advantageous prices aiming to maximize its utility, considering factors such as energy consumption and profits. ⑥ Each SP then sends the chosen price to SR through the trusted third party. The purchasing and pricing process is repeated until the utilities of SR and SP converge. After the above process is finished, SR sends the partition of input data to the selected SPs.

For instance, when the surveillance camera identifies the faces from a captured image using a Convolutional Neural Network (CNN)-based image classification model, it offloads a face recognition task to SPs since the camera cannot meet the delay requirements within its limited computing resources. Then, the camera first makes partitioning and offloading decisions as shown in Fig. 1. According to the partitioning and offloading decisions, SPs perform the feature extraction for the partitioned tasks as illustrated in Fig. 2. Specifically, the camera, SR₂ splits the input image and offloads the partitioned tasks of Task 2 to SP₂, SP₄, and SP_{|D|} for the feature extraction. As the partitions of the previous task, Task 1, are being processed at SP₄ and SP_{|D|}, the partitions of Task 2 must wait until the ongoing partitions of Task 1 are completed (depicted as gray bars). Once the process for partition of Task 1 at SP₄ is finished, SP₄ can start extracting features for the partition of Task 2 at the first layer. After the feature extraction is finished at the first layer, SP₄ waits for SP₂ and SP_{|D|} to process the other partitioned tasks at the first layer. Once all processing tasks have been completed for the first layer and the results are propagated, SPs start the tasks for the next layer. This process repeats until the partitioned tasks are processed at the last layer of the CNN. Subsequently, the result of CNN inference is transmitted to the camera.

Note that when inference tasks are newly requested, the framework makes the optimal partition and offloading decision in an online fashion with the consideration of previously requested tasks from SRs that are being processed, allowing multiple DNN tasks to be executed in parallel.

B. DNN Partitioning Model Across Distributed EDs

In this study, we focus on CNNs as they are widely used in a variety of applications including image classification and object detection.

Given a CNN task n , $\mathcal{L}_n = [1, 2, \dots, l, \dots, L_n]$ denotes the layers in order, where a layer l refers to a convolution (Conv) layer. Note that input data partitioning splits the data among EDs and performs the inference in a parallel manner to fully utilize the computing resources of each ED [4] [18]. Thus, for the CNN task n , we split the image data into partitions in a piece-wise manner, where the sizes of partitions are determined based on the capabilities of EDs ($\in \mathcal{D}$).

In this study, piece-wise partitioning is conducted along one dimension of the input image, the size of which is H , for the CNN task n . The piece-wise partitioning may prolong the inference latency due to transmissions between EDs when processing the input data across partition boundaries. For communication efficiency, each ED sends only the boundary rows of its output data of each layer to the other ED [3]. Let $s_{i,n,l}$ represent the boundary rows of output data from the l -th layer, transmitting from ED i . We denote $a_{i,n}$ as a partition assigned to ED i , which is the number of rows covered by the partition. When the CNN task n is split along the height H of input, $a_{i,n}$ is subject to the following constraints:

$$a_{i,n} \geq s_{i,n,1} I\{a_{i,n} > 0\}, \quad (1)$$

$$\sum_{i \in \mathcal{D}} a_{i,n} = H, \quad (2)$$

where $I\{a_{i,n} > 0\}$ is an indicative function that values 1 if $a_{i,n} > 0$, otherwise 0, signifying whether the partition contains data. In (1), the size of the partition on ED i is supposed to be no smaller than that of the boundary rows of convolutional layers to be sent to neighboring EDs of ED i . (2) implies that the length of all partitions, when concatenated along the height dimension, equals H .

C. Scheduling Delay Model

Let $\Phi_{i,j}$ be the bandwidth between EDs i and j ($i, j \in \mathcal{D}$). We can obtain the achievable transmission rate from the ED i to ED j as

$$b_{i,j} = \Phi_{i,j} \log_2(1 + \gamma_{i,j}) \quad (3)$$

where $\gamma_{i,j} = \frac{\rho_{i,j} h_{i,j}}{\sigma^2}$ is the signal to noise ratio (SNR), σ^2 is the average power of background noise, and $\rho_{i,j}$ and $h_{i,j}$ are the transmission power and the channel gain, respectively, between EDs i and j .

Let t_n be the time when a CNN task n ($n \in \mathcal{N}$) is requested by a user device d . Given the partition, $a_{i,n}$, the transmission time for the partition from the user device to ED i is expressed

as

$$T_{i,n}^{(tr)} = \frac{\beta_{n,1} a_{i,n}}{b_{d,i}} = t_{i,n}^{(tr)} - t_n, \quad (4)$$

where $\beta_{n,l}$ is the two dimensional area (in bytes) determined by multiplying the width by the number of channels of layer l ($1 \leq l \leq L_n$).

If the task n finds that the partitions of previously requested interference tasks, m , $a_{i,m}$ are being processed or waiting at ED i , then it ought to wait till the computation of all the previous partitions is complete, $a_{i,m}$ to be processed. Let $t_{i,n}^{(w)}$ be the time when the waiting of task n is over. Then, considering the partitions from the previously requested tasks, $a_{i,m}$, we have

$$t_{i,n}^{(w)} = \max_{m \in \mathcal{N}} \{t_{i,m,L_m}^{(f)} I\{t_n > t_m\} I\{a_{i,m} > 0\}\}. \quad (5)$$

Given the computing intensity, δ_i and frequency, f_i of ED i , we estimate the delay for processing the layer l of the partition on ED i as follows:

$$T_{i,n,l}^{(p)} = \frac{\beta_{n,l} \delta_i a_{i,n}}{f_i} = t_{i,n,l}^{(f)} - t_{i,n,l}^{(s)}, \quad (6)$$

where $t_{i,n,l}^{(s)}$ and $t_{i,n,l}^{(f)}$ represent the instants when processing the layer l ($1 \leq l \leq L_n$) of task n starts and finishes, respectively.

After a single convolutional layer operation in ED i , the boundary rows of the output data, $s_{i,n,l}$ are exchanged with the neighboring EDs for the next layer operation. This transmission time is given by

$$T_{i,n,l}^{(tr)} = \max_{i' \in \{j,k\}} \frac{\beta_{n,l} s_{i,n,l} I(a_{i,n} > 0)}{b_{i,i'}}, \quad (7)$$

where $\{j,k\}$ indicates the previous ED j or the next ED k of ED i .

Each ED i must wait until all its required input is received before starting the processing of layer l of the partition assigned to itself. This starting time is computed as

$$t_{i,n,l}^{(s)} = \begin{cases} \max_{i \in \mathcal{D}} \{t_{i,n}^{(w)}, t_{i,n}^{(tr)}\}, & \text{if } l = 1 \\ \max_{i \in \mathcal{D}} \{t_{i,n,l-1}^{(f)} + T_{i,n,l-1}^{(tr)}\}, & \text{otherwise,} \end{cases} \quad (8)$$

where for the case of $l = 1$, the processing of the first layer of task n can be started when the partition of input data, $a_{i,n}$ is received and all the previous partitions, $a_{i,m}$ are processed.

We then have the total delay for processing the inference task n as follows:

$$T_n = \max_{i \in \mathcal{D}} t_{i,n,L_n}^{(f)} - t_n \leq \tau_n, \quad (9)$$

where τ_n is the deadline constraint for the CNN task n .

IV. STACKELBERG GAME BASED PARTITIONING WITH SCHEDULING

In this section, we formulate the strategic interaction of SRs and SPs as a two-level Stackelberg game. Specifically, we design our system framework as a computing resource market that enables multiple SRs to purchase the computing resources from multiple SPs, and the SPs accommodate the interference

partitions requested by the SRs such that the profits of both SRs and SPs are maximized.

A. Partition and Scheduling Decision Model

Assuming that one SR requests one CNN task n , $|\mathcal{N}|$ SRs and $|\mathcal{D}|$ SPs participate in the trading process.

For each task n , each SP i first individually decides how much of the task will it process, that is the partition, $a_{i,n}$. For this, we introduce a continuous variable that decides how the task n is split along the height of the input, $x_{i,n}$ ($0 \leq x_{i,n} \leq 1$) as follows:

$$a_{i,n} = \begin{cases} \lfloor x_{i,n}H \rfloor, & \text{if } x_{i,n}H - \lfloor x_{i,n}H \rfloor < 0.5 \\ \lceil x_{i,n}H \rceil, & \text{otherwise} \end{cases}$$

$$\text{s. t. } \sum_{i \in \mathcal{D}} x_{i,n} = 1, \quad (10)$$

where $x_{i,n}$ signifies the proportion that the $a_{i,n}$ partition covers.

When the partition, $a_{i,n}$ is decided for SP i , it must be ensured that the task ends before the deadline τ_n through scheduling. That is, the inequality in (9) must be satisfied. We can first re-express (9) as

$$t_{i,n,L_n}^{(f)} - t_n \leq T_n \leq \tau_n. \quad (11)$$

From (6) and (8), we have

$$t_{i,n,l}^{(f)} \geq T_{i,n,l}^{(p)} + T_{i,n,l-1}^{(tr)} + t_{i,n,l-1}^{(f)} \quad \text{for } l > 1 \quad (12)$$

This can be expressed as

$$\sum_{l=2}^{L_n} (T_{i,n,l}^{(p)} + T_{i,n,l-1}^{(tr)}) + T_{i,n,1}^{(p)} + t_{i,n,1}^{(s)} \leq t_{i,n,L_n}^{(f)}. \quad (13)$$

Using (11) and (13), we obtain the following inequality:

$$t_{i,n,1}^{(s)} + \sum_{l=2}^{L_n} T_{i,n,l-1}^{(tr)} + \sum_{l=1}^{L_n} T_{i,n,l}^{(p)} - t_n \leq \tau_n. \quad (14)$$

Since $\sum_{l=1}^{L_n} T_{i,n,l}^{(p)} = \frac{a_{i,n} \sum_{l=1}^{L_n} \beta_{n,l} \delta_i}{f_i}$, from (14), we derive the the maximum value of $a_{i,n}$ (denoted by $\hat{a}_{i,n}$) such that the deadline constraint in (9) is met as follows:

$$a_{i,n} \leq \min \left\{ \left\lfloor \frac{(\tau_n + t_n - t_{i,n,1}^{(s)} - \sum_{l=2}^{L_n} T_{i,n,l-1}^{(tr)}) f_i}{\sum_{l=1}^{L_n} \beta_{n,l} \delta_i} \right\rfloor, H \right\} = \hat{a}_{i,n}. \quad (15)$$

Then, from (15), we obtain the following constraint for $x_{i,n}$ to satisfy the deadline constraint:

$$0 \leq x_{i,n} \leq \frac{\hat{a}_{i,n}}{H}, \quad (16)$$

where $\hat{x}_{i,n}$ denotes the constraint for $x_{i,n}$, $\frac{\hat{a}_{i,n}}{H}$.

B. Energy Cost

All the SPs contest the sharing of their own resources because finishing CNN tasks drain their energy. As in [35], under the assumption of a low CPU voltage, the power consumption of CPU is ζf_i^3 , where ζ is a constant depending on the switched

capacitance of ED. The energy consumption for processing the partition assigned to SP i is given by

$$E_{i,n}^{(p)} = \frac{w_{i,n}}{f_i} (\zeta f_i^3) + \sum_{l \in L_n} E_{i,l}^{(tr)}, \quad (17)$$

where $w_{i,n}$ is the workload size of the partition $a_{i,n}$, which is computed as $w_{i,n} = \sum_{l \in L_n} \beta_{n,l} \delta_i H x_{i,n}$.

If l is not the last layer for the task n (i.e. $l < L_n$), SP i only sends the padding data. Otherwise, it sends the inference result data of the task n , $\beta_{n,L_n+1} a_{i,n}$. Then, the energy consumption by sending the output data is given by

$$E_{i,l}^{(tr)} = \begin{cases} (\rho_{i,j} + \rho_{i,k}) T_{i,l}^{(tr)}, & \text{if } l < L_n \\ \frac{\rho_{i,n} \beta_{n,L_n+1} a_{i,n}}{b_{i,d}}, & \text{otherwise,} \end{cases} \quad (18)$$

where j and k indicate ED i 's previous and next EDs, respectively.

C. Strategies of Service Requester and Service Providers

In our distributed partition with scheduling game, the set of SPs decides their prices for each task requested by each SR, acting as a leader. As a follower, reacting to the leader's strategy, each SR devices its own strategy purchasing the resources from the SPs.

Each SP sets the price in the upper Stage I, while each SR decides which SP to buy the computational resources from in the lower Stage II, based on the price charged by the SP. We formulated the optimization problems for the leaders (i.e., SPs) and followers (i.e., SRs) as follows:

1) *SPs' Pricing Strategies in Stage I*: Let $p_{i,n}$ denote the unit price charged by SP i for the computing resource. Each SP's profit can be defined as the price received from the SRs minus the cost incurred for the energy consumption. Thus, for the partition $a_{i,n}$ of CNN task n (i.e., SR n), the utility function of SP i is expressed as

$$U_{i,n} = p_{i,n} w_{i,n} - \alpha E_{i,n}^{(p)}, \quad (19)$$

where α is the coefficient that tunes the weight on the SP's energy consumption.

The $|\mathcal{D}|$ SPs lead the pricing to maximize their utility $U_{i,n}$

Problem 1: (SP Subgame):

$$\begin{aligned} & \max_{p_{i,n}} U_{i,n}, \quad i \in \mathcal{D} \\ & \text{s.t. } C_1 : 0 \leq p_{i,n} \leq \hat{p}, \end{aligned} \quad (20)$$

where $\hat{p}$ is the maximum price constraint.

2) *SR's Competing Strategies in Stage II*: When the SR n decides to offload its partition $a_{i,n}$ to the SP i , it also has to pay the cost of energy consumption for offloading the partition, in addition to the price $p_{i,n}$ offered by the SP. Then, the SR's payment is given by

$$P_{i,n} = p_{i,n} w_{i,n}. \quad (21)$$

Let T_i denote the time required to compute the partition $a_{i,n}$ at SP i . By letting $x_{i,n} = 1$, T_i can be obtained as follows:

$$\begin{aligned} T_i &= \sum_{l=1}^{L_n} T_{i,n,l}^{(p)} + t_{i,n,1}^{(s)} - t_n \\ &= \frac{\sum_{l \in L_n} \beta_{n,l} \delta_i H}{f_i} + t_{i,n,1}^{(s)} - t_n. \end{aligned} \quad (22)$$

Using (22), we now define the SR's degree of satisfaction as an exponential utility function, by referring to [7], [36]:

$$g_{i,n} = \left(1 - e^{-\frac{x_{i,n}}{f_{i,n}}}\right) e, \quad (23)$$

where

$$f_{i,n} = \frac{T_i}{\tau_n}, \quad (24)$$

indicating that the SR's degree of satisfaction is determined by the ratio of time for the SP i processing a_i to the deadline τ . Then, the utility function for the SR is modeled such that it increases as $P_{i,n}$ decreases as follows:

$$\hat{U}_n = \sum_{i \in \mathcal{D}} (g_{i,n} - P_{i,n}). \quad (25)$$

Given the SPs' prices, the $|\mathcal{N}|$ SRs determine their purchase to maximize their utility, $\hat{U}_n$. Then the sub-game problem from the SRs' side can be written as follows:

Problem 2: (SR Subgame):

$$\begin{aligned} &\max_{\mathbf{x}_n} \hat{U}_n \\ \text{s.t. } &C_1 : (16), \\ &C_2 : \sum_{i \in \mathcal{D}} x_{i,n} = 1. \end{aligned} \quad (26)$$

V. EQUILIBRIUM ANALYSIS AND OPTIMAL STRATEGIES FOR PARTITIONING WITH SCHEDULING GAME

Problems 1 and 2 are presented in the previous section for the Stackelberg game. The goal of the game is to seek the Stackelberg equilibrium, which is the leaders' optimal strategy given that the followers play their best responses to the leaders. Thus, in this section, we analyze the existence and uniqueness of the Stackelberg equilibrium point of partitioning with the scheduling game formulated in the previous section. Then, the optimal strategies for both SRs and SPs participating in the game are decided within their strategy spaces: $\{\mathbf{x}_n = [x_{i,n}]_{i \in \mathcal{D}} : 0 \leq x_{i,n} \leq \hat{x}_{i,n}\}$ and $\{\mathbf{p}_n = [p_{i,n}]_{i \in \mathcal{D}} : 0 \leq p_{i,n} \leq \hat{p}\}$.

A. Stage II: SRs' Demand Game

Definition 1: Let $\mathbf{x}_n^*$ and $\mathbf{p}_n^*$ denote the optimal strategy for SR n and the best price strategies for all the SPs, respectively. Then, the point $(\mathbf{x}_n^*, \mathbf{p}_n^*)$ is the Stackelberg equilibrium if the following conditions are satisfied:

$$U_{i,n}(p_{i,n}^*, \mathbf{p}_{-i,n}^*, \mathbf{x}_n^*) \geq U_{i,n}(p_{i,n}, \mathbf{p}_{-i,n}^*, \mathbf{x}_n^*), \quad \forall i, \forall n \quad (27)$$

and

$$\hat{U}_n(\mathbf{x}_n^*, \mathbf{p}_n^*) \geq \hat{U}_n(\mathbf{x}_n, \mathbf{p}_n^*), \quad \forall n, \quad (28)$$

where $\mathbf{p}_{-i,n}^*$ is the optimal price strategy combination of other SPs for the SR n 's task.

Theorem 1: Given the SPs' pricing strategies $\mathbf{p}_n$, the Nash equilibrium exists for $x_{i,n}$.

Proof: To prove that the utility $\hat{U}_n(\mathbf{x}_n, \mathbf{p}_n)$ is a concave function, we first take the first-order and second-order derivatives of (25) with respect to $x_{i,n}$. Letting $W_n = \sum_{l \in L_n} \beta_{n,l} \delta_i H$, the derivatives are given by

$$\begin{aligned} \frac{\partial(\hat{U}_n(\mathbf{x}_n, \mathbf{p}_n))}{\partial x_{i,n}} &= \frac{1}{f_{i,n}} e^{(1-\frac{x_{i,n}}{f_{i,n}})} - \frac{\partial P_{i,n}}{\partial x_{i,n}} \\ &= \frac{1}{f_{i,n}} e^{(1-\frac{x_{i,n}}{f_{i,n}})} - p_{i,n} W_n, \end{aligned} \quad (29)$$

and

$$\frac{\partial^2(\hat{U}_n(\mathbf{x}_n, \mathbf{p}_n))}{\partial^2 x_{i,n}} = -\frac{1}{f_{i,n}^2} e^{(1-\frac{x_{i,n}}{f_{i,n}})}. \quad (30)$$

From (24), $\frac{\partial^2(\hat{U}_n(\mathbf{x}_n, \mathbf{p}_n))}{\partial^2 x_{i,n}}$ is negative. Therefore, we have proved that $\hat{U}_n(\mathbf{x}_n, \mathbf{p}_n)$ is strictly concave with respect to $x_{i,n}$. Accordingly, the Nash equilibrium exists in this SR subgame. The proof is hence complete. $\square$

From Theorem 1, the Slater's condition holds for the optimization problem in the SR subgame (i.e., (26)). Hence, we can obtain the unique optimal solution for $x_{i,n}$ by applying Karush-Kuhn-Tucker (KKT) conditions.

Let $\eta^{(1)}, \eta_i^{(2)}, \eta_i^{(3)}$ denote Lagrangian multipliers. Then the Lagrangian of the optimization problem given in (26) can be represented as

$$\begin{aligned} L(x_{1,n}, \dots, x_{|\mathcal{D}|,n}, \eta^{(1)}, \eta_1^{(2)}, \dots, \eta_{|\mathcal{D}|}^{(2)}, \eta_1^{(3)}, \dots, \eta_{|\mathcal{D}|}^{(3)}) \\ = - \sum_{i \in \mathcal{D}} \left(e - e^{(1-\frac{x_{i,n}}{f_{i,n}})} - P_{i,n} \right) - \eta^{(1)} \left(\sum_{i \in \mathcal{D}} x_{i,n} - 1 \right) \\ - \sum_{i \in \mathcal{D}} \eta_i^{(2)} x_{i,n} + \sum_{i \in \mathcal{D}} \eta_i^{(3)} (x_{i,n} - \hat{x}_{i,n}), \end{aligned} \quad (31)$$

where $\eta^{(1)} \in \mathbb{R}, \eta_i^{(2)} \geq 0, \eta_i^{(3)} \geq 0, (i \in \mathcal{D})$. Then, the KKT conditions of the problem in (31) are presented as follows:

$$\frac{\partial L}{\partial x_{i,n}} = p_{i,n} W_n - \frac{1}{f_{i,n}} e^{(1-\frac{x_{i,n}}{f_{i,n}})} - \eta^{(1)} - \eta_i^{(2)} + \eta_i^{(3)} = 0 \quad (32)$$

$$\frac{\partial L}{\partial \eta^{(1)}} = \sum_{i \in \mathcal{D}} x_{i,n} - 1 = 0 \quad (33)$$

$$\frac{\partial L}{\partial \eta_i^{(2)}} = x_{i,n} = 0 \quad (34)$$

$$\frac{\partial L}{\partial \eta_i^{(3)}} = x_{i,n} - \hat{x}_{i,n} = 0 \quad (35)$$

$$\eta_i^{(2)}, \eta_i^{(3)} \geq 0. \quad (36)$$

By solving the above KKT conditions of (32)–(35), the optimal solution, $x_{i,n}^*$ for SP i can be obtained in the following theorem.

Theorem 2: The unique Nash equilibrium for SR n in the SR subgame is given by

$$x_{i,n}^* = \begin{cases} 0, & \text{if } p_{i,n}W_n - \frac{e}{f_{i,n}} > \eta^{(1)}. \\ \hat{x}_{i,n}, & \text{if } p_{i,n}W_n - \frac{e^{(1-\frac{\hat{x}_{i,n}}{f_{i,n}})}}{f_{i,n}} < \eta^{(1)} \\ f_{i,n} (1 - \ln f_{i,n} (p_{i,n}W_n - \eta^{(1)})), & \text{otherwise.} \end{cases} \quad (37)$$

Proof: The optimal solution takes one of the following three forms:

Case 1: $\eta_i^{(2)} = 0, \eta_i^{(3)} = 0$: Equation (32) can be represented as

$$\eta^{(1)} = p_{i,n}W_n - \frac{1}{f_{i,n}} e^{(1-\frac{x_{i,n}}{f_{i,n}})}. \quad (38)$$

From the above equation, we get the following optimal solution:

$$x_{i,n}^* = f_{i,n} \left(1 - \ln \left(p_{i,n}W_n - \eta^{(1)} \right) - \ln f_{i,n} \right) \quad (39)$$

Case 2: $\eta_i^{(2)} \neq 0, \eta_i^{(3)} = 0$: From (32), we get $p_{i,n}W_n - \frac{1}{f_{i,n}} e^{1-\frac{x_{i,n}}{f_{i,n}}} = \eta^{(1)} + \eta_i^{(2)}$. Since $\eta^{(2)} \neq 0$, to satisfy (34), $x_{i,n}^*$ must be zero. Thus, we have

$$p_{i,n}W_n - \frac{e}{f_{i,n}} = \eta^{(1)} + \eta_i^{(2)} > \eta^{(1)}. \quad (40)$$

Case 3: $\eta_i^{(2)} = 0, \eta_i^{(3)} \neq 0$. To satisfy (35) in this case, $x_{i,n}^*$ is represented as $x_{i,n}^* = \hat{x}_{i,n}$. Accordingly, from (35), we can derive that $p_{i,n}W_n - \frac{1}{f_{i,n}} e^{(1-\frac{\hat{x}_{i,n}}{f_{i,n}})} = \eta^{(1)} - \eta_i^{(3)}$. Therefore, SP i , which fulfills the (37) has the partition vector, $x_{i,n} = \hat{x}_{i,n}$.

$$p_{i,n}W_n - \frac{1}{f_{i,n}} e^{(1-\frac{\hat{x}_{i,n}}{f_{i,n}})} = \eta^{(1)} - \eta_i^{(3)} < \eta^{(1)}. \quad (41)$$

□

We now compute the best response, $x_{i,n}^*$ derived in Theorem 2 by using the BEST-REPLY algorithm, which is presented in Algorithm 1.

As expressed in (37), partition vector $x_{i,n}^*$ depends on the variable $\eta^{(1)}$. In Algorithm 1, it iteratively determines the value of $\eta^{(1)}$ that satisfies $\sum_{i \in \mathcal{D}} x_{i,n}^* = 1$ to obtain the optimal value for $x_{i,n}^*$ (Line 1). First, using (40), the algorithm finds the devices satisfying Case 2, denoted as $x_{i,n}^* = 0$ (Line 2). It then examines the devices that satisfy Case 3, denoted as $x_{i,n}^* = \hat{x}_{i,n}$ (Line 3). Finally, for Case 1, the algorithm applies (39) to get $x_{i,n}^*$ (Line 11). Therefore, the BEST-REPLY algorithm can determine the optimal partition vector that satisfies the KKT condition.

The BEST-REPLAY algorithm finds an $\eta^{(1)}$ that satisfies $h(\eta^{(1)}) = 0$ for $h(\eta^{(1)}) = \sum_{i \in \mathcal{D}} x_{i,n}^* - 1$ using the bisection method, as in [37]. We now prove that the BEST-REPLAY algorithm can obtain $\eta^{(1)}$ by the following lemma:

Lemma 1: Given that $\sum_{i \in \mathcal{D}} \hat{x}_{i,n} \geq 1$, $h(\eta^{(1)})$ has only one root in the range of $[\eta^{(\min)}, \eta^{(\max)}]$, where

Algorithm 1: BEST-REPLY Algorithm.

Input: SR's satisfaction degree $f_{i,n}$, first-order derivative $P'_{i,n}$

Output: $x_{i,n}^*$

- 1: Compute $\eta^{(1)}$ in $h(\eta^{(1)}) = 0$ using bisection method in the range of $[\eta^{(\min)}, \eta^{(\max)}]$
 - 2: **for** $i = 1; i \leq |\mathcal{D}|; i++$ **do**
 - 3: **if** $\eta^{(1)} < p_{i,n}W_n - \frac{e}{f_{i,n}}$ **then**
 - 4: # Case 2 as $\eta^{(1)}$ satisfies (40)
 - 5: $x_{i,n}^* \leftarrow 0$
 - 6: **else if** $\eta^{(1)} > p_{i,n}W_n - \frac{1}{f_{i,n}} e^{(1-\frac{\hat{x}_{i,n}}{f_{i,n}})}$ **then**
 - 7: # Case 3 as $\eta^{(1)}$ satisfies (41)
 - 8: $x_{i,n}^* \leftarrow \hat{x}_{i,n}$
 - 9: **else**
 - 10: # Case 1 as (40) and (41) are dissatisfied.
 - 11: $x_{i,n}^* \leftarrow f_{i,n}(1 - \ln(p_{i,n}W_n - \eta) - \ln f_{i,n})$
 - 12: **end if**
 - 13: **end for**
 - 14:
 - 15: **return** $x_{i,n}^*$ for $\forall i$
-

$$\eta^{(\min)} = \min_{i \in \mathcal{D}} \{p_{i,n}W_n - \frac{e}{f_{i,n}}\} \text{ and } \eta^{(\max)} = \max_{i \in \mathcal{D}} \{p_{i,n}W_n - \frac{1}{f_{i,n}} e^{1-\frac{\hat{x}_{i,n}}{f_{i,n}}}\}.$$

Proof: $x_{i,n}^*$ in (37) has to meet the conditions in (16) and (32). Thus, $\sum_{i \in \mathcal{D}} \hat{x}_{i,n} \geq \sum_{i \in \mathcal{D}} x_{i,n}^* = 1$. This proves that $\sum_{i \in \mathcal{D}} \hat{x}_{i,n} \geq 1$.

Let $\mathcal{D}_1$, $\mathcal{D}_2$ and $\mathcal{D}_3$ denote the subsets of $\mathcal{D}$ that are determined by the three cases in (37). Then, we can redefine the set $\mathcal{D}$ as the following subsets, $\mathcal{D}_1 = \{i_1 | i_1 \in \mathcal{D}, P'_{i_1,n} - \frac{e}{f_{i_1,n}} \geq \eta^{(1)}\}$, $\mathcal{D}_2 = \{i_2 | i_2 \in \mathcal{D}, P'_{i_2,n} - \frac{1}{f_{i_2,n}} e^{(1-\frac{\hat{x}_{i_2,n}}{f_{i_2,n}})} \leq \eta^{(1)}\}$, and $\mathcal{D}_3 = \mathcal{D} - \mathcal{D}_1 - \mathcal{D}_2$. In terms of these three sets, $h(\eta^{(1)})$ is rewritten as

$$h(\eta^{(1)}) = \sum_{i_1 \in \mathcal{D}_1} 0 + \sum_{i_2 \in \mathcal{D}_2} \hat{x}_{i_2,n} + \sum_{i_3 \in \mathcal{D}_3} x_{i_3,n} - 1. \quad (42)$$

The first-order derivative of $h(\eta^{(1)})$ in (42) with respect to $\eta^{(1)}$ is

$$\frac{\partial h(\eta^{(1)})}{\partial \eta^{(1)}} = \sum_{i_3 \in \mathcal{D}_3} \frac{f_{i_3,n}}{p_{i_3,n}W_n - \eta^{(1)}} > 0. \quad (43)$$

Therefore, $h(\eta^{(1)})$ is an increasing function.

If $\eta^{(1)} = \eta^{(\min)}$, it holds that $p_{i,n}W_n - f_{i,n}e \geq \eta^{(1)}$ for $\forall i$. It follows that $\mathcal{D} = \mathcal{D}_1$ and $\mathcal{D}_2 = \mathcal{D}_3 = \emptyset$. Hence, we have

$$h(\eta^{(\min)}) = \sum_{i_1 \in \mathcal{D}} 0 - 1 < 0. \quad (44)$$

If $\eta^{(1)} = \eta^{(\max)}$, it holds that $P'_{i_2,n} - \frac{1}{f_{i_2,n}} e^{(1-\frac{\hat{x}_{i_2,n}}{f_{i_2,n}})} \leq \eta^{(1)}$, for $\forall i$, which follows $\mathcal{D} = \mathcal{D}_2$ and $\mathcal{D}_1 = \mathcal{D}_3 = \emptyset$. Then,

Algorithm 2: Repeated Stackelberg Game.

Input: Select initial input $\mathbf{p} = [p_{i,n}]_{i \in \mathcal{D}}$ where $p_{i,n} \leftarrow [0, \hat{p}]$, precision threshold ϵ , gradient step size μ
Output: Optimal price $\mathbf{p}_n^*$, optimal partitioning strategy $\mathbf{x}_n^*$
Initialize: $\mathbf{p}' \leftarrow \mathbf{0}$
1: **while** $\frac{\|\mathbf{p} - \mathbf{p}'\|_1}{\|\mathbf{p}\|_1} < \epsilon$ **do**
2: Obtain $\mathbf{x}_n^*$ from $\mathbf{p}$ using Algorithm 1.
3: $\mathbf{p}' \leftarrow \mathbf{p}$
4: **for** $i = 1; i \leq D; i++$ **do**
5: # Update the price, $\mathbf{p}[i]$, according to the derivative direction to maximize the utility of SP i , $U_{i,n}$.
6: $\mathbf{p}[i] \leftarrow \mathbf{p}'[i] + \mu \frac{\partial U_{i,n}}{\partial p_{i,n}}$
7: **end for**
8: **end while**

we have

$$h(\eta^{(\max)}) = \sum_{i_2 \in \mathcal{D}} \hat{x}_{i_2,n} - 1 > 0. \quad (45)$$

Therefore, we can say that the function $h(\eta^{(1)})$ has only one root in the interval $[\eta^{(\min)}, \eta^{(\max)}]$. $\square$

1) *Stage I: SPs' Utility Maximization:* Considering the optimal strategy $\mathbf{x}_n^*$ in Stage II, each SP decides its optimal pricing strategy in Stage I by solving the problem that maximizes its utility in (20).

Theorem 3: Given the optimal strategy $\mathbf{x}_n^*$, the SP's optimal price $p_{i,n}$ can be uniquely obtained.

Proof: We prove that $U_{i,n}$ is concave with respect to $p_{i,n}$ by calculating $\frac{\partial U_{i,n}}{\partial p_{i,n}}$ and demonstrating $\frac{\partial^2 U_{i,n}}{\partial p_{i,n}^2} < 0$ in Appendix, available online. $\square$

According to Theorem 3, we know that $U_{i,n}(p_{i,n}, \mathbf{p}_{-i,n}, \mathbf{x}_n^*)$ has a unique solution. The concave function, having a unique solution, converges to the optimal solution as the price $p_{i,n}$ is adjusted according to the derivative direction. Thus, we can find the optimal price $\mathbf{p}_n^*$ using the gradient-based search algorithm, as shown in Algorithm 2.

VI. PERFORMANCE EVALUATION

In this section, we detail the simulation experiments conducted to verify the efficiency and effectiveness of our proposed algorithm in terms of the total delay required to finish DNN inference tasks, as well as the energy consumption and profits of SPs.

A. Simulation Setup

In the simulation, three different types of SPs, ED₁, ED₂, and ED₃ have been considered, representing desktop PCs with i9-13900 k processors, laptops/smartphones, and IoT devices (e.g. smart speaker), respectively [11], [38], [39].

Simulations are conducted on an area of 400 × 400 m, each of which is evenly divided into 16 squares (100 × 100 m). In the center of each square, 16 ED₁ SPs are deployed, while one ED₂ and one ED₃ SPs are randomly distributed within a maximum distance of 50 m from each ED₁ [14], [36]. On the other hand,

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Computing intensity (δ_i) [3]	1100 cycles per bytes
Number of tasks ($ \mathcal{N} $)	$[16 \times 60, 48 \times 60]$
Delay constraint (τ_n) [3]	200 ms
Gaussian noise (σ^2) [18], [31]	-60 dbm
Transmission bandwidth (Φ) [31], [32]	20 MHz
Transmission power ($\rho_{i,j}$) [31], [32]	1 W
Energy constant (ζ) [35]	$2 \cdot 10^{-28}$
Energy coefficient (α)	0.1, 0.2 ¹
Maximum price ($\bar{p}$)	10

TABLE III
SPECIFICATION OF THREE DEVICE TYPES

Device Type	Device's Model	Frequency (f_i)
ED ₁	Desktop PC (i9-13900k)	5 GHz
ED ₂	Smartphone (iPhone 14 pro)	3 GHz
ED ₃	IoT (Jetson Nano)	2 GHz

SRs are randomly distributed in the considered area, and each SR may communicate with a varying number of SPs based on its location.

The total number of DNN inference tasks from SRs during the simulation varies from 1000 to 2500 task requests, with an increment of 500. The inference services randomly request one of the three widely used DNN models: VGG-16, AlexNet, and Network in Network (NiN). The price is charged for the 10^9 CPU cycles of workloads. The main parameters used in the simulation are presented in Table II.

We compare our proposed IPO scheme with two benchmark methods, IPO-R (Random) and IPO-G (Genetic Algorithm). In IPO-R, each SP randomly decides the price for computing resources from zero to the maximum price constraint, $\hat{p}$, while IPO-G employs GA. Then, the partition and scheduling decisions are made using the BEST-REPLY algorithm.

B. Simulation Results

Fig. 3 illustrates the convergence performance of IPO as the number of iterations increases when $\alpha = 0.1$ and 0.2. It can be observed that the utilities of SR and SP converge and remain stable after 50 iterations for both α values. Specifically, as shown in Fig. 3(a) and (b), the utility of SR decreases, while that of SP increases until 50 iterations. This is because the prices of all SPs initially start from the minimum price. Hence, SRs can send the workloads to SPs with low payments, resulting in low profits of SPs at the beginning of iterations. Consequently, as the iterations progress, SPs increase their prices to optimize their utility leading to an increase in the utility of SP but a decrease in the utility of SR. We can also observe that ED₁ has the highest utility because its price is higher than that of ED₂ and ED₃. When $\alpha = 0.2$, both SR and SP utilities decrease because the price of computing resources increases as shown in Fig. 6. Hence,

¹To satisfy $x_{i,n}^* > 0$, it should hold that $\frac{\partial U_{i,n}}{\partial x_{i,n}} > 0$. From (19) and (29), we have $p_{i,n} > \alpha \zeta f_i^2$ and $p_{i,n} < \frac{e}{f_{i,n} W_n} < \frac{e f_i \tau_n}{W_n^2}$, which gives $0 < \alpha \leq \min_{n \in \mathcal{N}, i \in \mathcal{D}} \frac{e \tau_n}{\zeta f_i W_n^2} = 0.2$.

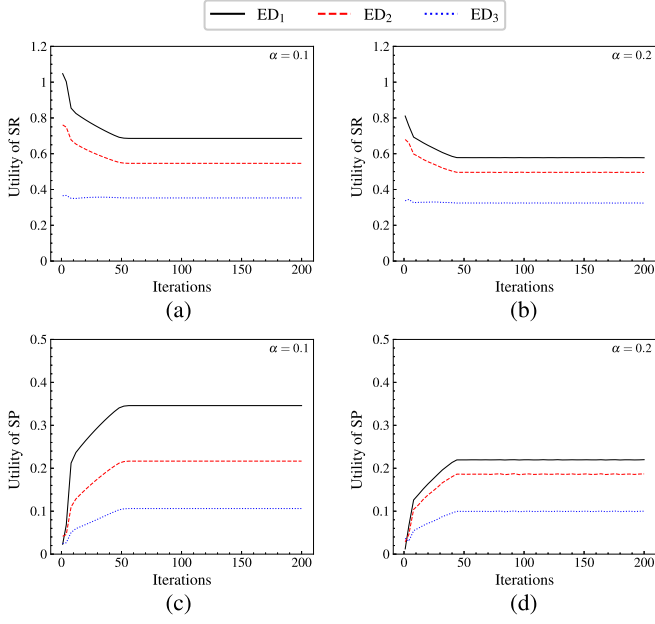
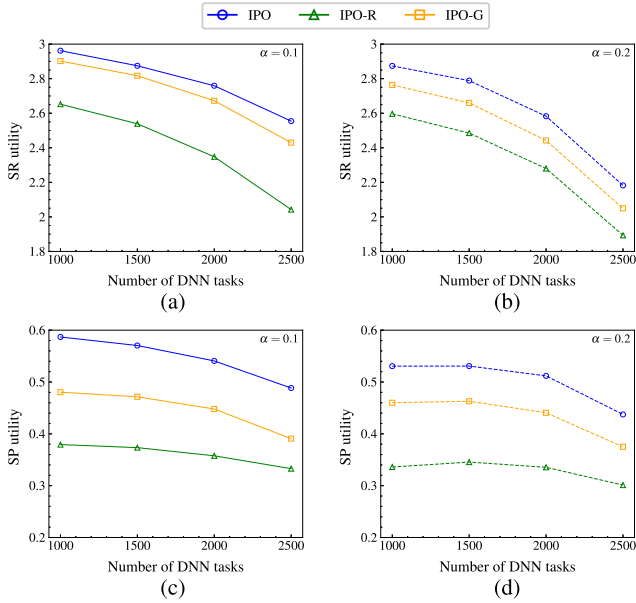
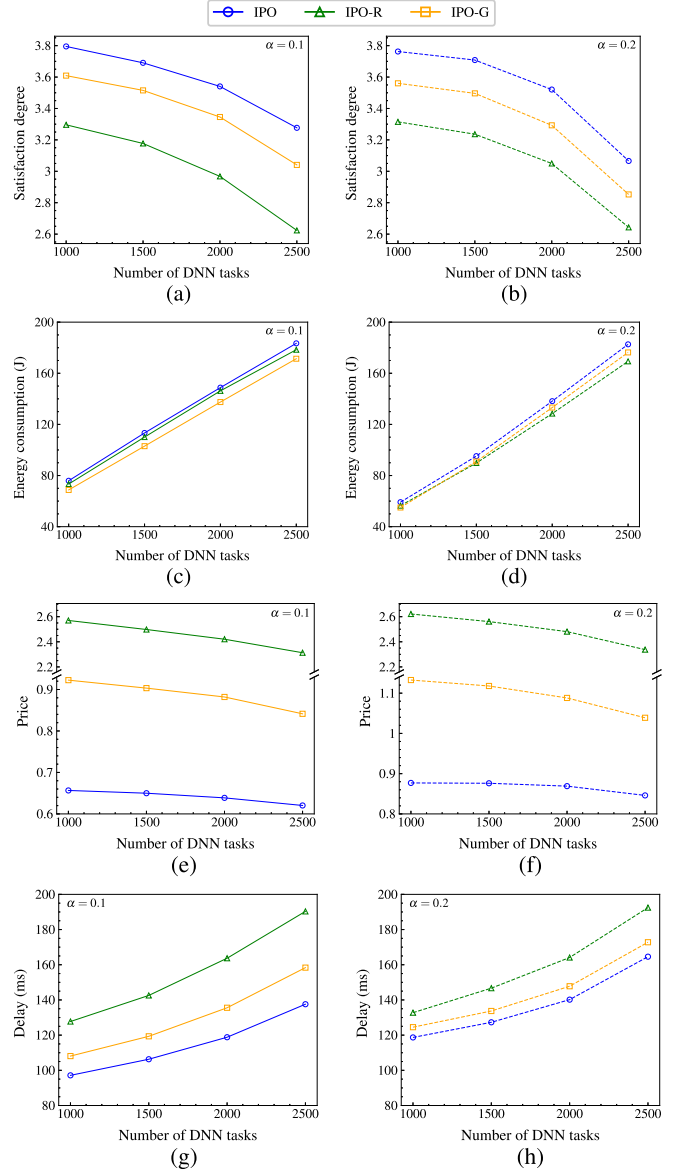


Fig. 3. Convergence of the SR and SP utility.

Fig. 4. SR and SP utility with respect to the number of DNN tasks. (a)-(b) SR utility when $\alpha = 0.1$ and 0.2. (c)-(d) SP utility when $\alpha = 0.1$ and 0.2.

to minimize their payments, SRs send more workloads to ED₃ which offer the lowest prices but have the lowest computing resources, resulting in low user satisfaction and lower utility of SR. This also reduces the profits of SPs leading to lower SP utility compared to that when $\alpha = 0.1$.

Fig. 4 plots the utility of SR and SP with respect to the number of DNN tasks. Fig. 4 shows that the proposed method, denoted by IPO, outperforms IPO-R and IPO-G in terms of the utility of SR and SP for both values of $\alpha = 0.1$ and 0.2. We can observe that both utilities of SR and SP decrease as the number of DNN tasks increase because a higher number of requests leads to increased energy consumption as well as

Fig. 5. Performance comparisons with respect to the number of DNN tasks. (a)-(b) Satisfaction degrees of SRs when $\alpha = 0.1$ and 0.2. (c)-(d) Energy consumptions of SPs when $\alpha = 0.1$ and 0.2. (e)-(f) Prices of computing resources when $\alpha = 0.1$ and 0.2. (g)-(h) Total inference delays when $\alpha = 0.1$ and 0.2.

decreased user satisfaction, as depicted in Fig. 5. Compared to IPO-R and IPO-G, when $\alpha = 0.1$, the proposed IPO achieves a higher average utility of SRs by 16.88 % and 3.13 % (51.35 % and 22.22 % for SPs), respectively. IPO shows the highest utility of SR because it achieves a higher satisfaction degree for SR, as depicted in Fig. 5(a) and (b). This is because, compared to IPO-G, SRs in IPO send more workloads to ED₁, which has higher computing resources than ED₂ and ED₃, resulting in lower inference delay, as depicted in Fig. 6. However, because more workloads are sent to ED₁ which consumes more energy than ED₂ and ED₃, IPO shows higher energy consumption than IPO-G. IPO-R sends more workloads to ED₃ and ED₂ compared to IPO-R and IPO-G, resulting in lower inference delay and SR utility than IPO.

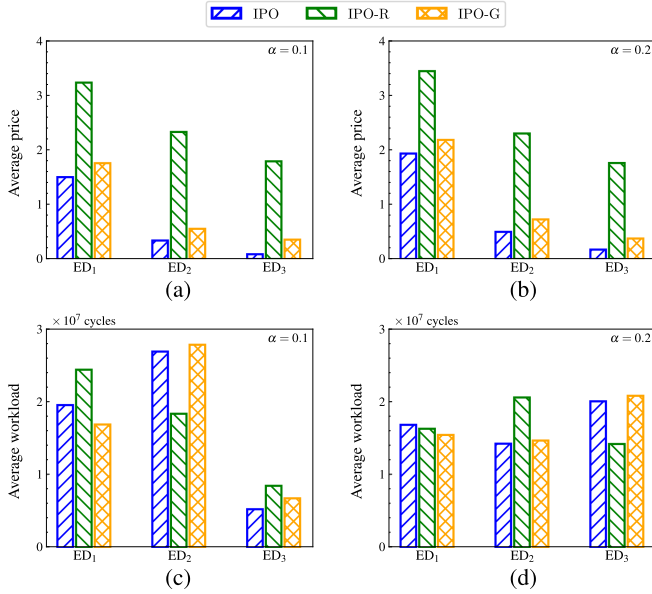


Fig. 6. Average price and workload on each type of device.

Despite IPO having higher energy consumption, as indicated in Fig. 5(c) and (d), the SPs in IPO achieve higher utility compared to IPO-R and IPO-G. This is because the utility of the SPs is not only determined by energy consumption, but it also influenced by pricing strategies. In IPO, SPs engage in competition with each other, resulting in the establishment of a Nash equilibrium in pricing. Hence, the prices of SPs are set to optimize the average profits of SPs. On the other hand, in IPO-G, each SP independently sets its price to optimize its utility without considering the prices set by other devices, resulting in higher average prices than IPO, as depicted in Fig. 5(e) and (f). In IPO and IPO-G, the prices of SPs are determined by considering the partition and offloading decisions made by SRs. However, in IPO-R, SPs randomly decide the price from zero to maximum price ($\hat{p}$) without considering the partition and offloading decisions leading to higher average prices than in IPO and IPO-G.

In IPO, the lower price of ED₁ in IPO compared to that in IPO-G and IPO-R cause SRs to send more workloads to ED₁, as shown in Fig. 6(c) and (d), recalling that $w_{i,n}$ is the workload size of $a_{i,n}$. Because the utility of SP is also affected by the workload of each device, the higher workloads on ED₁, which has the highest prices, result in greater SP utility in IPO, compared to IPO-G and IPO-R.

In contrast to the case of $\alpha = 0.1$, when $\alpha = 0.2$, the utility of the SP decreases as SR sends a higher number of workloads to ED₃ compared to ED₁ due to higher prices of the latter as shown in Fig. 4, resulting in low energy consumption of SPs. However, the decrease in profits of ED₁ leads to a reduction in the utility of the SP. Although a higher workload on ED₃ results in lower energy consumption, this also reduces user satisfaction due to its lower computing resources compared to ED₁, as depicted in Fig. 5, leading to a decrease in the SR utility. Nevertheless, IPO outperforms IPO-R and IPO-G in terms of utility when $\alpha = 0.2$. Specifically, compared to IPO-R and IPO-G, IPO achieves 12.86 % and 5.27 % higher utility for SRs on average

(52.35 % and 15.70 % for SPs), compared to IPO-R and IPO-G, respectively.

Fig. 5 demonstrates the average satisfaction degree of SRs, energy consumption of SPs, average price of computing resources, and total inference delay with respect to the number of DNN tasks when $\alpha = 0.1$ and 0.2. Fig. 5 (a) and (b) depict that the satisfaction degree decreases as the number of DNN tasks increases for all three algorithms due to increment of inference delay. Fig. 5 (c) and (d) illustrate that more DNN tasks incoming to SPs also cause higher energy consumption due to processing of more tasks. When the value of $\alpha = 0.2$, an increment in prices resulting from higher energy consumption leads to a lower satisfaction degree than in the case of $\alpha = 0.1$. Fig. 5 (e) and (f) show that average price decreases as the number of tasks increases for all three algorithms. An increase in the number of DNN tasks results in a rise in waiting delay at SPs, as any new tasks must wait until the previously requested tasks are completed as shown in Fig. 2. Consequently, the inference delay also increases as illustrated in Fig. 5 (g) and (h), leading to a decrease in the average prices offered by SPs. Specifically, when $\alpha = 0.1$, prices decrease by 5.52 %, 10.00 %, and 8.80 % as the number of DNN tasks increases from 1000 to 2500 for IPO, IPO-R, and IPO-G, respectively. Fig. 5 (g) and (h) show that IPO achieves lower total inference delay than IPO-R and IPO-G. As expressed in (5)–(9), IPO makes the offloading and scheduling decision based on the finish time of each partition by considering the previously requested interference tasks in the EDs, resulting in lower total inference delay for parallel execution compared to that achieved using IPO-R and IPO-G.

Fig. 6 shows the average prices of computing resources and average workload on each SP type (i.e., the sum of workloads on each type of SP over the total number of tasks) for the three algorithms. When $\alpha = 0.1$, all algorithms send more workloads to ED₁ and ED₂ than to ED₃ as they offer greater computing resources at relatively lower prices. However, when $\alpha = 0.2$, the prices of all device types increase, with the price increment for ED₁ and ED₂ being larger than that in the case of ED₃, as shown in Fig. 6(a) and (b). Therefore, SRs are less likely to send workloads to ED₁ and ED₂ but instead, send more workloads to ED₃ to minimize their payments, as depicted in Fig. 6(c) and (d).

As shown in Figs. 4, 5, and 6, the prices of SPs are influenced by the energy consumption of SPs and waiting time of the offloaded tasks. Specifically, a higher energy consumption reduces the utility of SPs, resulting in increased prices. In contrast, extended waiting times contribute to increased inference delays, leading to a decrease in the prices of SPs. Increased inference delays reduce the satisfaction degrees of SRs, motivating SPs to decrease their prices to encourage SRs to continue offloading tasks.

VII. CONCLUSION

In this paper, we proposed an incentive-aware partitioning and offloading (IPO) scheme that motivates edge devices to cooperate for multiple DNN inference task executions in a parallel manner. To provide an incentive mechanism for edge devices, we modeled the edge computing environment as a computing resource market, where DNN inference service requesters

(SR) pay service providers (SPs) for using their computing resources. Furthermore, we developed the DNN partitioning and pricing strategies that consider the task dependencies for parallel execution to maximize the utility of both SPs and SRs. Considering the interaction between SRs and SPs as a two-level Stackelberg game in the market, DNN partitioning and pricing strategies are independently and dynamically determined by each SP and SR. Finally, the simulation results confirmed that the proposed IPO outperforms other benchmark algorithms in terms of the utility of SR and SP by optimizing the DNN partitioning and pricing decision compared to IPO-Random (IPO-R) and IPO-GA (IPO-G).

REFERENCES

- [1] H. Zhou, M. Li, N. Wang, G. Min, and J. Wu, "Accelerating deep learning inference via model parallelism and partial computation offloading," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 2, pp. 475–488, Feb. 2023.
- [2] X. Chen, J. Zhang, B. Lin, Z. Chen, K. Wolter, and G. Min, "Energy-efficient offloading for DNN-Based smart IoT systems in cloud-edge environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 683–697, Mar. 2022.
- [3] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, Apr. 2021.
- [4] X. Zhang, Z. Li, C. Lai, and J. Zhang, "Joint edge server placement and service placement in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11 261–11 274, Jul. 2022.
- [5] A. H. Sodhro, S. Pirbhulal, and V. H. C. de Albuquerque, "Artificial intelligence-driven mechanism for edge computing-based industrial applications," *IEEE Trans. Ind. Inform.*, vol. 15, no. 7, pp. 4235–4243, Jul. 2019.
- [6] M. Xue, H. Wu, G. Peng, and K. Wolter, "DDPQN: An efficient DNN offloading strategy in local-edge-cloud collaborative environments," *IEEE Trans. Serv. Comput.*, vol. 15, no. 2, pp. 640–655, Mar./Apr. 2022.
- [7] T. Wang, Y. Lu, J. Wang, H.-N. Dai, X. Zheng, and W. Jia, "EIHPD: Edge-intelligent hierarchical dynamic pricing based on cloud-edge-client collaboration for IoT systems," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1285–1298, Aug. 2021.
- [8] T. Kim, H. Park, Y. Jin, S.-S. Lee, and S. Lee, "Partition placement and resource allocation for multiple DNN-based applications in heterogeneous IoT environments," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9836–9848, Jun. 2023.
- [9] B. Kar, W. Yahya, Y.-D. Lin, and A. Ali, "A survey on offloading in federated cloud-edge-fog systems with traditional optimization and machine learning," 2022. [Online]. Available: <https://arxiv.org/abs/2202.10628>
- [10] W. Fan, L. Gao, Y. Su, F. Wu, and Y. Liu, "Joint DNN partition and resource allocation for task offloading in Edge-cloud-assisted IoT environments," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10146–10159, Jun. 2023.
- [11] P. Lin, Z. Shi, Z. Xiao, C. Chen, and K. Li, "Latency-driven model placement for efficient edge intelligence service," *IEEE Trans. Services Comput.*, vol. 15, no. 2, pp. 591–601, Mar./Apr. 2022.
- [12] C. Rong, J. H. Wang, J. Liu, J. Wang, F. Li, and X. Huang, "Scheduling massive camera streams to optimize large-scale live video analytics," *IEEE/ACM Trans. Netw.*, vol. 30, no. 2, pp. 867–880, Apr. 2022.
- [13] P. Ren, X. Qiao, Y. Huang, L. Liu, C. Pu, and S. Dustdar, "Fine-grained elastic partitioning for distributed DNN towards mobile web AR services in the 5G era," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3260–3274, Nov./Dec. 2022.
- [14] Y. Xu, T. Mohammed, M. Di Francesco, and C. Fischione, "Distributed assignment with load balancing for DNN inference at the edge," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1053–1065, Jan. 2023.
- [15] M. A. Dhuheir, E. Baccour, A. Erbad, S. S. Al-Obaidi, and M. Hamdi, "Deep reinforcement learning for trajectory path planning and distributed inference in resource-constrained UAV swarms," *IEEE Internet Things J.*, vol. 10, no. 9, pp. 8185–8201, May 2023.
- [16] W. Shi, S. Zhou, Z. Niu, M. Jiang, and L. Geng, "Multiuser co-inference with batch processing capable edge server," *IEEE Trans. Wirel. Commun.*, vol. 22, no. 1, pp. 286–300, Jan. 2023.
- [17] W. He, S. Guo, S. Guo, X. Qiu, and F. Qi, "Joint DNN partition deployment and resource allocation for delay-sensitive deep learning inference in IoT," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9241–9254, Oct. 2020.
- [18] F. Zeng, Q. Chen, L. Meng, and J. Wu, "Volunteer assisted collaborative offloading and resource allocation in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3247–3257, Mar. 2021.
- [19] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient acceleration of deep learning inference on resource-constrained edge devices: A review," *Proc. IEEE*, vol. 111, no. 1, pp. 42–91, Jan. 2023.
- [20] J. Huang, M. Wang, Y. Wu, Y. Chen, and X. Shen, "Distributed offloading in overlapping areas of mobile-edge computing for Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13837–13847, Aug. 2022.
- [21] J. Chen, H. Wu, R. Li, and P. Jiao, "Green parallel online offloading for DSCI-Type tasks in IoT-Edge systems," *IEEE Trans. Ind. Inform.*, vol. 18, no. 11, pp. 7955–7966, Nov. 2022.
- [22] X. Chen, M. Li, H. Zhong, Y. Ma, and C.-H. Hsu, "DNNOff: Offloading DNN-Based intelligent IoT applications in mobile edge computing," *IEEE Trans. Ind. Inform.*, vol. 18, no. 4, pp. 2820–2829, Apr. 2022.
- [23] J. R. J. Liu, Y. Zhang and Y. Zhang, "Auction-based dependent task offloading for IoT users in edge clouds," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 4907–4921, Mar. 2023.
- [24] J. Xue, Q. Hu, Y. An, and L. Wang, "Joint task offloading and resource allocation in vehicle-assisted multi-access edge computing," *Comput. Commun.*, vol. 177, pp. 77–85, Sep. 2021.
- [25] J. Tang, W. Zhu, X. Li, S. Liu, and X. Liu, "A distributed pricing strategy for edge computation offloading optimization in autonomous driving," *IEEE Netw.*, early access, Aug. 8 2022, doi: [10.1109/MNET.122.2200119](https://doi.org/10.1109/MNET.122.2200119).
- [26] S. Deng, Y. Chen, G. Chen, S. Ji, J. Yin, and A. Y. Zomaya, "Incentive-driven proactive application deployment and pricing on distributed edges," *IEEE Trans. Mob. Comput.*, vol. 22, no. 2, pp. 951–967, Feb. 2023.
- [27] M. Diamanti, P. Charatsaris, E. E. Tsiropoulou, and S. Papavassiliou, "Incentive mechanism and resource allocation for edge-fog networks driven by multi-dimensional contract and game theories," *IEEE Open J. Commun. Soc.*, vol. 3, pp. 435–452, 2022.
- [28] W. Fan et al., "DNN deployment, Task offloading, and resource allocation for joint task inference in IIoT," *IEEE Trans. Ind. Inform.*, vol. 19, no. 2, pp. 1634–1646, Feb. 2023.
- [29] G. Qu, H. Wu, R. Li, and P. Jiao, "DMRO: A deep meta reinforcement learning-based task offloading framework for edge-cloud computing," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 3448–3459, Sep. 2021.
- [30] G. Yoon, G.-Y. Kim, H. Yoo, S. C. Kim, and R. Kim, "Implementing practical DNN-Based object detection offloading decision for maximizing detection performance of mobile edge devices," *IEEE Access*, vol. 9, pp. 140199–140211, 2021.
- [31] Y. Chen, S. Zhang, Y. Jin, Z. Qian, and S. Lu, "Multi-server multi-user game at edges for heterogeneous video analytics," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 841–846.
- [32] W. Hou, H. Wen, N. Zhang, J. Wu, W. Lei, and R. Zhao, "Incentive-driven task allocation for collaborative edge computing in industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 706–718, Jan. 2022.
- [33] G. Baranwal, D. Kumar, and D. P. Vidyarthi, "BARA: A blockchain-aided auction-based resource allocation in edge computing enabled Industrial Internet of Things," *Future Gener. Comput. Syst.*, vol. 135, pp. 333–347, Oct. 2022.
- [34] Y. Su, W. Fan, Y. Liu, and F. Wu, "A truthful combinatorial auction mechanism towards mobile edge computing in Industrial Internet of Things," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1678–1691, Apr./Jun. 2023.
- [35] C. You, K. Huang, and H. Chae, "Energy efficient mobile cloud computing powered by wireless energy transfer," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 5, pp. 1757–1771, May. 2016.
- [36] R. Roostaee, Z. Dabiri, and Z. Movahedi, "A game-theoretic joint optimal pricing and resource allocation for mobile edge computing in NOMA-based 5G networks and beyond," *Comput. Netw.*, vol. 198, Oct. 2021, Art. no. 108352.
- [37] T. Zhang, K.-Y. Lam, J. Zhao, F. Li, H. Han, and N. Jamil, "Enhancing federated learning with spectrum allocation optimization and device selection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 5, pp. 1981–1996, Oct. 2023.
- [38] F. Li, H. Yao, J. Du, C. Jiang, and Y. Qian, "Stackelberg game-based computation offloading in social and cognitive Industrial Internet of Things," *IEEE Trans. Ind. Inform.*, vol. 16, no. 8, pp. 5444–5455, Aug. 2020.
- [39] G. Li and J. Cai, "An online incentive mechanism for collaborative task offloading in mobile edge computing," *IEEE Trans. Wirel. Commun.*, vol. 19, no. 1, pp. 624–636, Jan. 2020.



TaeYoung Kim received the bachelor's degree in computer science from the Yonsei University, Korea, in 2016. He is currently working toward the PhD degree in computer science with the Yonsei University. He is under the supervision of Prof. SuKyoung Lee. His research interests include wireless networks and edge computing.



Seung-seob Lee received the BS and PhD degrees in computer science from Yonsei University, Seoul, South Korea, in 2011, and 2019. He is currently postdoctoral researcher with the Department of Computer Science, Yale University. His research interests include the fields of fog/edge computing, especially for machine learning based resource optimization and system-level network security for heterogeneous fog/edge devices.



Chang Kyung Kim received the BS degree in computer science from the University of Rochester, USA, in 2019, and the MS degree in computer science from Yonsei University, South Korea, in 2021. He is currently working toward the PhD degree with Yonsei University. His research interests include focus on the performance of edge computing and network systems.



SuKyoung Lee (Member, IEEE) received the BS, MS, and PhD degrees in computer science from Yonsei University, Seoul, South Korea, in 1992, 1995, and 2000, respectively. From 2000 to 2003, she was with the Advanced Networking Technologies Division, National Institute of Standards and Technology, Gaithersburg, Maryland. She is currently a professor with the Department of Computer Science, Yonsei University, Seoul, South Korea. Her current research interests include span wireless networking, mobile/edge systems, and vehicular networks